

Data Structures & Algorithms

Masterclass: Structures, Complexity Analysis & Problem Solving

Presented by: Asad Usman

1. Introduction & Complexity Analysis

An algorithm is a step-by-step procedure for solving a computational problem, while data structures provide systematic ways to organize and store data. Evaluating their efficiency requires asymptotic analysis.

Why Complexity Matters: Scalability determines whether an algorithm handles millions of records gracefully or crashes due to exponential time expansion.

Asymptotic Notations

- **Big-O (O):** Upper bound (worst-case performance guarantee).
- **Big-Omega (Ω):** Lower bound (best-case execution metric).
- **Big-Theta (Θ):** Tight bound (average or exact growth rate).

Common Growth Classes

- **$O(1)$ - Constant:** Array index lookup, hash table access.
- **$O(\log n)$ - Logarithmic:** Binary search in sorted collections.
- **$O(n)$ - Linear:** Single loop traversal through unindexed data.
- **$O(n \log n)$ - Linearithmic:** Efficient comparison-based sorts (Merge Sort, Heap Sort).
- **$O(n^2)$ - Quadratic:** Nested loops (Bubble Sort, Selection Sort).

2. Linear Data Structures

Linear data structures arrange elements sequentially where each element is connected to its previous and next element.

Arrays & Dynamic Arrays

Contiguous memory allocations offering $O(1)$ random access.

- **Fixed Array:** Rigid sizing declared at compile time.
- **Dynamic Array:** Automatically resizes (e.g., C++ `std::vector`, Python `list`) via amortization.

Linked Lists

Non-contiguous node chains containing data and pointer references.

- **Singly & Doubly Linked:** Supports flexible insertion/deletion ($O(1)$ at head) but requires $O(n)$ traversal.

Stacks (LIFO)

Last-In, First-Out structure.

- **Operations:** `push()`, `pop()`, `peek()` in $O(1)$ time.
- **Applications:** Expression evaluation, function call stacks, undo mechanisms.

Queues (FIFO)

First-In, First-Out structure.

- **Operations:** `enqueue()`, `dequeue()` in $O(1)$ time.
- **Applications:** BFS traversal, printer queues, task scheduling.

3. Non-Linear Data Structures (Trees & Graphs)

Non-linear structures represent hierarchical or networked relationships among data elements.

Trees & Binary Search Trees (BST)

- **Binary Tree:** Hierarchical structure where each node has at most two children (left and right).
- **Binary Search Tree (BST):** Ordered tree property where left subtree values $\leq$ parent < right subtree values.
- **Balanced Trees:** AVL Trees and Red-Black Trees maintain $O(\log n)$ height boundaries during insertions/deletions.

Graphs

Comprised of vertices (nodes) V and edges E , directed or undirected, weighted or unweighted.

- **Representations:** Adjacency Matrix vs. Adjacency List.
- **Traversals:** Breadth-First Search (BFS - queue-based) and Depth-First Search (DFS - stack/recursion-based).
- **Shortest Path & MST:** Dijkstra's Algorithm, Bellman-Ford, Kruskal's, and Prim's algorithms.

4. Advanced Problem Solving & Paradigms

Mastering algorithm design paradigms enables efficient problem-solving across complex engineering challenges.

Paradigm	Core Concept & Mechanism	Classic Examples
Divide & Conquer	Break problem into subproblems, solve recursively, combine results.	Merge Sort, Quick Sort, Binary Search
Dynamic Programming	Solve overlapping subproblems once, store results (Memoization / Tabulation).	Fibonacci, Knapsack, Longest Common Subsequence
Greedy Algorithms	Make locally optimal choices at each stage to find global optimum.	Activity Selection, Huffman Coding, Dijkstra
Backtracking	Build candidates incrementally and abandon paths violating constraints.	N-Queens, Sudoku Solver, Graph Coloring

Problem-Solving Methodology: Always clarify constraints, analyze edge cases, trace sample inputs on paper before coding, and evaluate time-space trade-offs.